🌿 CSIRO Pasture Biomass Estimation — Phase 3 & 4

Deep Learning Subject Project | Image2Biomass Kaggle Challenge

Objective: Predict 5 biomass targets (Dry_Green_g, Dry_Dead_g, Dry_Clover_g, GDM_g, Dry_Total_g)
Metric: Weighted R^2 on log-transformed targets
Weights: Green=0.1, Dead=0.1, Clover=0.1, GDM=0.2, Total=0.5

Architecture Overview

- **Backbone:** EfficientNet-B3 (pretrained on ImageNet)
- **Neck:** Multi-scale feature pooling
- **Head:** Multi-task regression with auxiliary metadata inputs
- **Training:** Cosine annealing LR, label smoothing, heavy augmentation
- **Ensemble:** Multiple folds + TTA (Test Time Augmentation)

1. Install Dependencies & Mount Drive

```
# Install required packages
!pip install -q timm albumentations efficientnet_pytorch
!pip install -q scikit-learn pandas numpy matplotlib seaborn tqdm
```

```
  Preparing metadata (setup.py) ... done
  Building wheel for efficientnet_pytorch (setup.py) ... done
```

```
from google.colab import drive
drive.mount('/content/drive')

DATA_PATH = "/content/drive/MyDrive/Deep Learning/csiro-biomass"
import os
print("Files:", os.listdir(DATA_PATH))
```

```
Mounted at /content/drive
Files: ['test.csv', 'sample_submission.csv', 'train.csv', 'train', '.ipynb_checkpoints', 'test', 'outputs']
```

2. Imports & Configuration

```
import os, gc, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from tqdm import tqdm
warnings.filterwarnings('ignore')

import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import timm
import albumentations as A
from albumentations.pytorch import ToTensorV2
from sklearn.model_selection import KFold
from sklearn.metrics import r2_score

# ── CONFIG ──────────────────────────────────────────────────────────
CFG = dict(
    seed         = 42,
```

```
    data_path    = DATA_PATH,
    img_size     = 384,          # larger resolution → better features
    backbone     = 'efficientnet_b3',  # stronger backbone than ResNet18
    pretrained   = True,
    n_folds      = 5,
    train_folds  = [0, 1, 2, 3, 4],
    batch_size   = 16,
    num_workers  = 2,
    epochs       = 25,
    lr           = 3e-4,
    min_lr       = 1e-6,
    weight_decay = 1e-4,
    warmup_epochs= 3,
    targets      = ['Dry_Green_g','Dry_Dead_g','Dry_Clover_g','GDM_g','Dry_Total_g'],
    weights      = [0.1, 0.1, 0.1, 0.2, 0.5],   # competition metric weights
    dropout      = 0.3,
    tta_steps    = 5,           # number of TTA augmentations
    use_metadata = True,        # use NDVI + height as auxiliary features
    grad_clip    = 1.0,
    device       = 'cuda' if torch.cuda.is_available() else 'cpu',
)

def set_seed(seed):
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)

set_seed(CFG['seed'])
print(f"Device: {CFG['device']}")
print(f"Backbone: {CFG['backbone']}")
```

```
Device: cuda
Backbone: efficientnet_b3
```

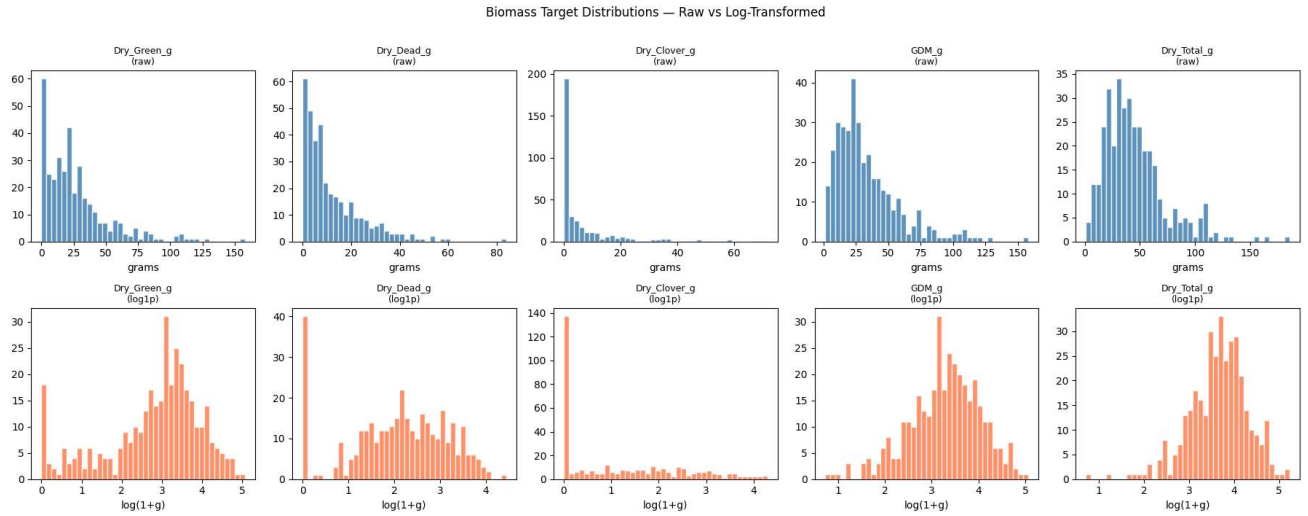
3. Data Loading & Preprocessing (EDA)

```
# ── FIXED Data Loading ──────────────────────────────────────────────
csv_path = os.path.join(DATA_PATH, 'train.csv')
df_raw = pd.read_csv(csv_path)
print("Raw shape:", df_raw.shape)
print("Columns:", df_raw.columns.tolist())
print("Sample target_names:", df_raw['target_name'].unique())
print("Sample targets (first 10):", df_raw['target'].head(10).values)

# ── CORRECT PIVOT: sample_id contains image ID + target, so strip it ─────────
# sample_id format: "ID1011485656__Dry_Green_g"
# We need to extract the base image ID first
df_raw['img_id'] = df_raw['sample_id'].str.split('__').str[0]

# Pivot using img_id (not sample_id) as index
meta_cols = ['img_id', 'image_path', 'Sampling_Date', 'State',
             'Species', 'Pre_GSHH_NDVI', 'Height_Ave_cm']

df = df_raw.pivot_table(
    index=meta_cols,
    columns='target_name',
    values='target',
    aggfunc='first'
).reset_index()
df.columns.name = None

print(f"\nAfter pivot shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")

# Verify all targets exist
for t in CFG['targets']:
    if t not in df.columns:
        print(f"  ⚠️  Missing column: {t}")
    else:
```

```
        print(f"  ✓ {t}: mean={df[t].mean():.2f}, max={df[t].max():.2f}, zeros={( df[t]==0).sum()}")

# Clean up
df = df.drop_duplicates(subset='image_path').reset_index(drop=True)
for col in CFG['targets']:
    df[col] = df[col].fillna(0).clip(lower=0)

print(f"\nFinal unique images: {len(df)}")
print(df[CFG['targets']].describe())
```

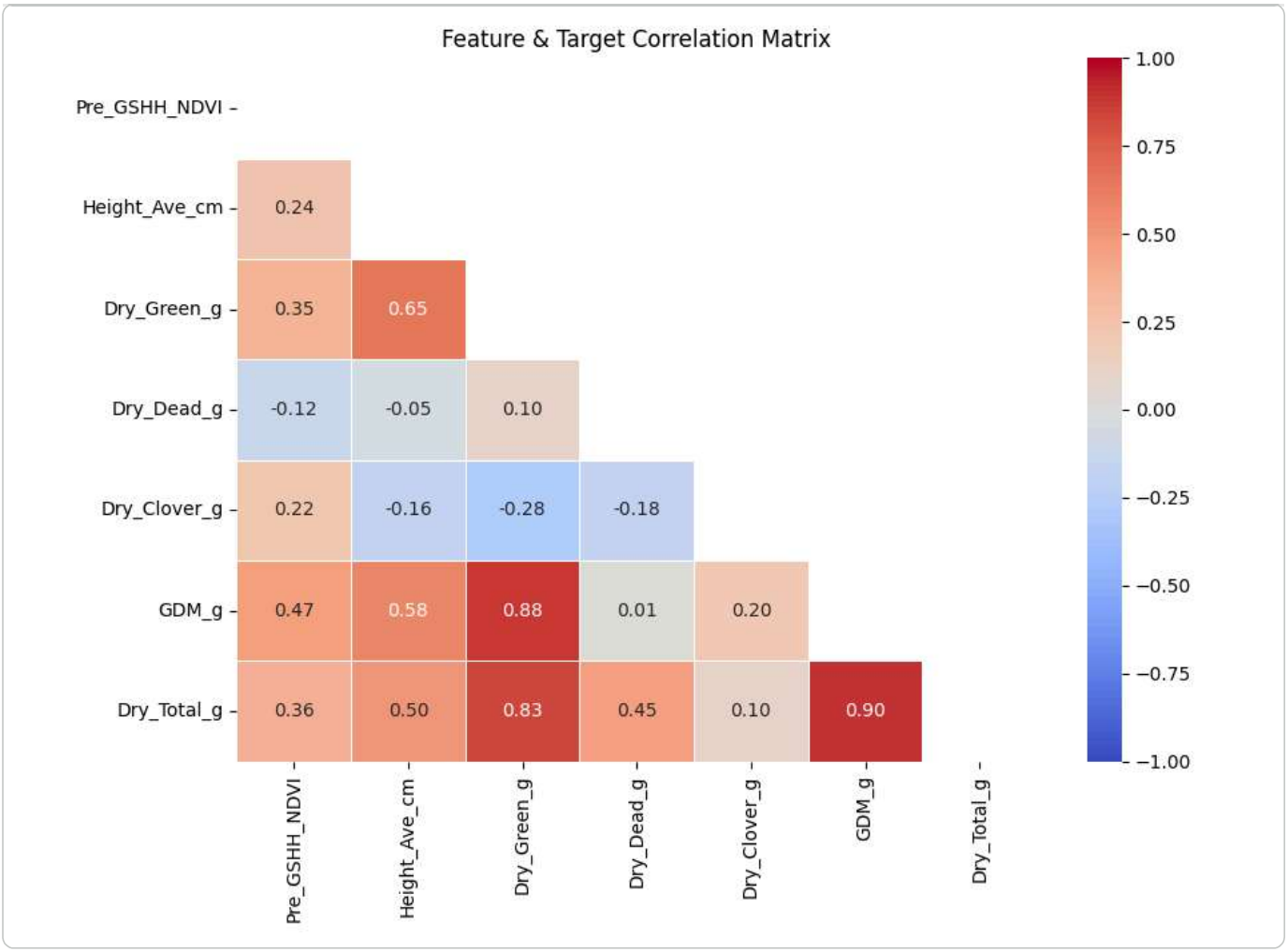
```
Raw shape: (1785, 9)
Columns: ['sample_id', 'image_path', 'Sampling_Date', 'State', 'Species', 'Pre_GSHH_NDVI', 'Height_Ave_cm', 'target_nam
Sample target_names: ['Dry_Clover_g' 'Dry_Dead_g' 'Dry_Green_g' 'Dry_Total_g' 'GDM_g']
Sample targets (first 10): [ 0.     31.9984 16.2751 48.2735 16.275   0.      0.      7.6     7.6
  7.6   ]

After pivot shape: (357, 12)
Columns: ['img_id', 'image_path', 'Sampling_Date', 'State', 'Species', 'Pre_GSHH_NDVI', 'Height_Ave_cm', 'Dry_Clover_g'
  ✓ Dry_Green_g: mean=26.62, max=157.98, zeros=18
  ✓ Dry_Dead_g: mean=12.04, max=83.84, zeros=40
  ✓ Dry_Clover_g: mean=6.65, max=71.79, zeros=135
  ✓ GDM_g: mean=33.27, max=157.98, zeros=0
  ✓ Dry_Total_g: mean=45.32, max=185.70, zeros=0

Final unique images: 357
       Dry_Green_g  Dry_Dead_g  Dry_Clover_g       GDM_g  Dry_Total_g
count   357.000000  357.000000    357.000000  357.000000   357.000000
mean     26.624722   12.044548      6.649692   33.274414    45.318097
std      25.401232   12.402007     12.117761   24.935822    27.984015
min       0.000000    0.000000      0.000000    1.040000     1.040000
25%       8.800000    3.200000      0.000000   16.026100    25.271500
50%      20.800000    7.980900      1.423500   27.108200    40.300000
75%      35.083400   17.637800      7.242900   43.675700    57.880000
max     157.983600   83.840700     71.786500  157.983600   185.700000
```

```
# ── EDA: Target distributions ──────────────────────────────────────
fig, axes = plt.subplots(2, 5, figsize=(18, 7))

for i, col in enumerate(CFG['targets']):
    axes[0, i].hist(df[col], bins=40, color='steelblue', edgecolor='white', alpha=0.85)
    axes[0, i].set_title(f'{col}\n(raw)', fontsize=9)
    axes[0, i].set_xlabel('grams')

    log_vals = np.log1p(df[col])
    axes[1, i].hist(log_vals, bins=40, color='coral', edgecolor='white', alpha=0.85)
    axes[1, i].set_title(f'{col}\n(log1p)', fontsize=9)
    axes[1, i].set_xlabel('log(1+g)')

plt.suptitle('Biomass Target Distributions — Raw vs Log-Transformed', fontsize=12, y=1.01)
plt.tight_layout()
plt.savefig('target_distributions.png', bbox_inches='tight', dpi=120)
plt.show()
print("Saved: target_distributions.png")
```

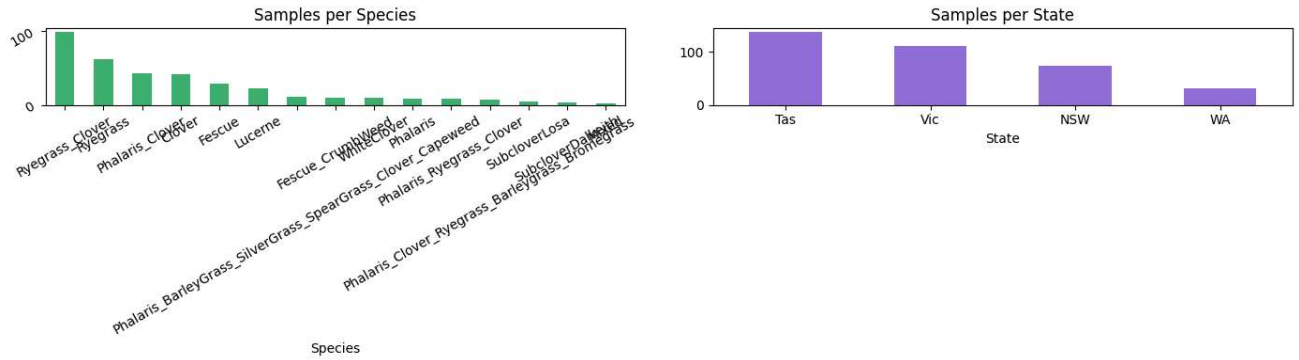



```
Saved: target_distributions.png
```

```
# ── EDA: Correlation heatmap ──────────────────────────────────────
corr_cols = ['Pre_GSHH_NDVI', 'Height_Ave_cm'] + CFG['targets']
corr = df[corr_cols].corr()

plt.figure(figsize=(9, 7))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, vmin=-1, vmax=1)
plt.title('Feature & Target Correlation Matrix')
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=120)
plt.show()
```




```
# ── EDA: Species distribution ──────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
df['Species'].value_counts().plot(kind='bar', ax=axes[0], color='mediumseagreen')
axes[0].set_title('Samples per Species'); axes[0].tick_params(rotation=30)
df['State'].value_counts().plot(kind='bar', ax=axes[1], color='mediumpurple')
axes[1].set_title('Samples per State'); axes[1].tick_params(rotation=0)
plt.tight_layout()
plt.savefig('species_state_distribution.png', dpi=120)
plt.show()
```




```
# ── Apply log transform to targets ─────────────────────────────────
for col in CFG['targets']:
    df[f'{col}_log'] = np.log1p(df[col])

log_targets = [f'{c}_log' for c in CFG['targets']]

# ── Metadata normalization ─────────────────────────────────────────
df['Pre_GSHH_NDVI'] = df['Pre_GSHH_NDVI'].fillna(df['Pre_GSHH_NDVI'].median())
```

```
df['Height_Ave_cm'] = df['Height_Ave_cm'].fillna(df['Height_Ave_cm'].median())
ndvi_mean, ndvi_std     = df['Pre_GSHH_NDVI'].mean(), df['Pre_GSHH_NDVI'].std()
height_mean, height_std = df['Height_Ave_cm'].mean(), df['Height_Ave_cm'].std()
df['ndvi_norm']   = (df['Pre_GSHH_NDVI'] - ndvi_mean)   / (ndvi_std   + 1e-6)
df['height_norm'] = (df['Height_Ave_cm'] - height_mean) / (height_std + 1e-6)

# ── Add K-Fold column ──────────────────────────────────────────────
kf = KFold(n_splits=CFG['n_folds'], shuffle=True, random_state=CFG['seed'])
df['fold'] = -1
for fold, (_, val_idx) in enumerate(kf.split(df)):
    df.loc[val_idx, 'fold'] = fold

print(df['fold'].value_counts().sort_index())
print("Sample row:\n", df[['image_path','fold','Pre_GSHH_NDVI','Height_Ave_cm'] + CFG['targets']].head(2).to_string())
```

```
fold
0    72
1    72
2    71
3    71
4    71
Name: count, dtype: int64
Sample row:
                image_path  fold  Pre_GSHH_NDVI  Height_Ave_cm  Dry_Green_g  Dry_Dead_g  Dry_Clover_g   GDM_g  Dry_Tota
0  train/ID1011485656.jpg     3           0.62         4.6667      16.2751     31.9984           0.0  16.275      48.27
1  train/ID1012260530.jpg     4           0.55        16.0000       7.6000      0.0000           0.0   7.600       7.60
```

4. Dataset & Augmentation Pipeline

```
# ── Albumentations augmentation pipelines ──────────────────────────
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

# ── FIX: Updated transforms compatible with Albumentations >= 1.4 ──────────
# Newer versions require size as a tuple (height, width), not a single int

def get_train_transforms(img_size):
    sz = (img_size, img_size)   # ← tuple required by newer albumentations
    return A.Compose([
        A.RandomResizedCrop(sz, scale=(0.7, 1.0), ratio=(0.9, 1.1)),
        A.HorizontalFlip(p=0.5),
        A.VerticalFlip(p=0.5),
        A.RandomRotate90(p=0.5),
        A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.1, rotate_limit=15, p=0.5),
        A.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.05, p=0.7),
        A.OneOf([
            A.GaussNoise(var_limit=(10, 50)),
            A.GaussianBlur(blur_limit=(3, 5)),
            A.MotionBlur(),
        ], p=0.3),
        A.CoarseDropout(max_holes=8, max_height=32, max_width=32, p=0.3),
        A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
        ToTensorV2(),
    ])

def get_val_transforms(img_size):
    sz = (img_size, img_size)
    return A.Compose([
        A.Resize(sz[0], sz[1]),
        A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
        ToTensorV2(),
    ])

def get_tta_transforms(img_size):
    sz = (img_size, img_size)
    return A.Compose([
        A.Resize(sz[0], sz[1]),
        A.HorizontalFlip(p=0.5),
        A.VerticalFlip(p=0.5),
        A.RandomRotate90(p=0.5),
        A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
```

```
        ToTensorV2(),
    ])

print("Transforms fixed for Albumentations >= 1.4  ✓")
```

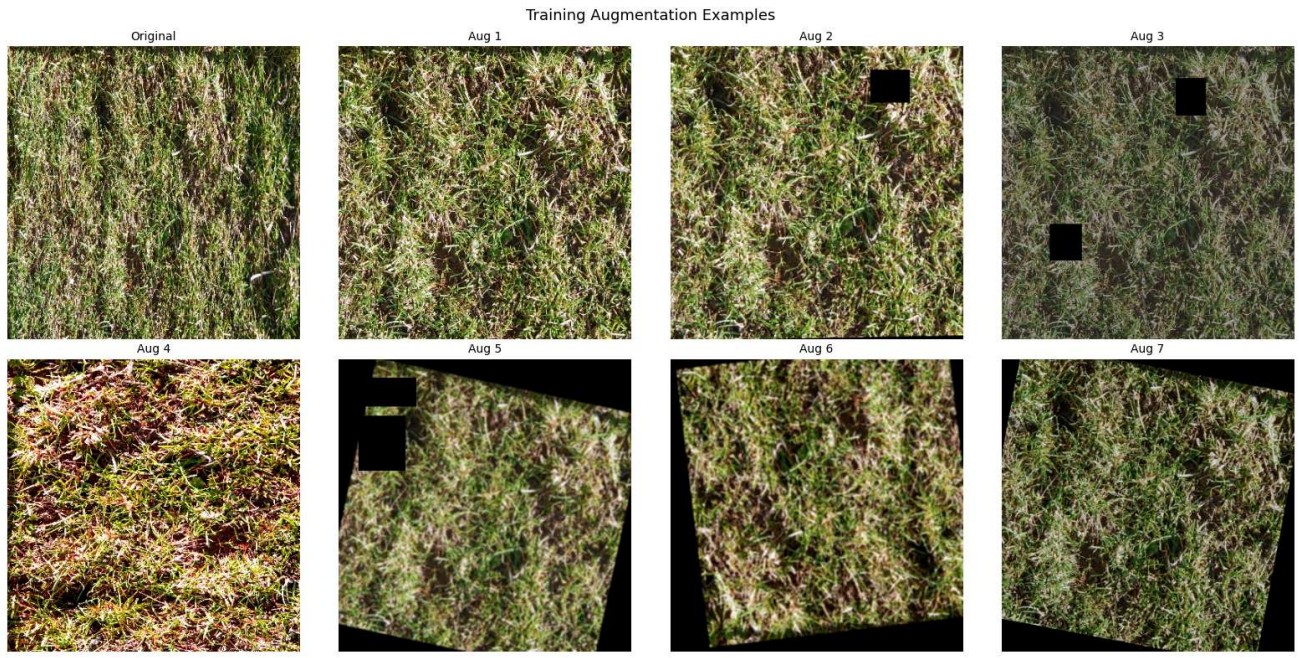
```
Transforms fixed for Albumentations >= 1.4  ✓
```

```
class BiomassDataset(Dataset):
    def __init__(self, df, data_path, transform=None, use_metadata=True, is_test=False):
        self.df           = df.reset_index(drop=True)
        self.data_path    = data_path
        self.transform    = transform
        self.use_metadata = use_metadata
        self.is_test      = is_test
        self.log_targets  = [f'{c}_log' for c in CFG['targets']]

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]

        # Load image
        img_path = os.path.join(self.data_path, row['image_path'])
        image = cv2.imread(img_path)
        if image is None:
            image = np.zeros((CFG['img_size'], CFG['img_size'], 3), dtype=np.uint8)
        else:
            image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        if self.transform:
            image = self.transform(image=image)['image']

        # Metadata features: [NDVI, height]
        if self.use_metadata and not self.is_test:
            meta = torch.tensor([
                row.get('ndvi_norm', 0.0),
                row.get('height_norm', 0.0)
            ], dtype=torch.float32)
        else:
            meta = torch.zeros(2, dtype=torch.float32)

        if self.is_test:
            return image, meta

        # Log-transformed labels
        labels = torch.tensor(
            row[self.log_targets].values.astype(np.float32)
        )
        return image, meta, labels

print("BiomassDataset class defined.")
```

```
BiomassDataset class defined.
```

```
# ── Visualise augmentations ──────────────────────────────────────
sample_row = df.sample(1).iloc[0]
img_path   = os.path.join(DATA_PATH, sample_row['image_path'])
img_bgr    = cv2.imread(img_path)
img_rgb    = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

aug = get_train_transforms(CFG['img_size'])

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes[0, 0].imshow(cv2.resize(img_rgb, (CFG['img_size'], CFG['img_size'])))
axes[0, 0].set_title('Original', fontsize=10); axes[0, 0].axis('off')

for i in range(1, 8):
    r, c = divmod(i, 4)
    aug_img = aug(image=img_rgb)['image']
    # Denormalise for display
```

```
    mean = np.array(IMAGENET_MEAN)[:, None, None]
    std  = np.array(IMAGENET_STD)[:, None, None]
    disp = (aug_img.numpy() * std + mean).transpose(1, 2, 0).clip(0, 1)
    axes[r, c].imshow(disp)
    axes[r, c].set_title(f'Aug {i}', fontsize=10); axes[r, c].axis('off')

plt.suptitle('Training Augmentation Examples', fontsize=13)
plt.tight_layout()
plt.savefig('augmentation_examples.png', dpi=120)
plt.show()
```



5. Model Architecture — EfficientNet-B3 with Metadata Fusion

```
class BiomassModel(nn.Module):
    """
    EfficientNet-B3 backbone + metadata fusion → multi-task regression head.
    Architecture:
      1. CNN backbone extracts 1536-d visual features
      2. Metadata MLP produces 32-d embedding
      3. Concatenation → shared FC → 5 target outputs
    """
    def __init__(self, backbone_name='efficientnet_b3', n_targets=5,
                 metadata_dim=2, dropout=0.3, pretrained=True):
        super().__init__()

        # ── Visual backbone ──────────────────────────────────────────
        self.backbone = timm.create_model(
            backbone_name,
            pretrained=pretrained,
            num_classes=0,       # remove classifier head
            global_pool='avg',
        )
        feat_dim = self.backbone.num_features  # 1536 for B3

        # ── Metadata MLP ─────────────────────────────────────────────
        self.meta_mlp = nn.Sequential(
            nn.Linear(metadata_dim, 64),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(64, 32),
            nn.ReLU(),
        )
```

```
        # ── Fusion head ──────────────────────────────────────────────
        fusion_dim = feat_dim + 32
        self.head = nn.Sequential(
            nn.Linear(fusion_dim, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(dropout / 2),
            nn.Linear(256, n_targets),
        )

    def forward(self, images, meta):
        img_feat  = self.backbone(images)          # (B, feat_dim)
        meta_feat = self.meta_mlp(meta)            # (B, 32)
        fused     = torch.cat([img_feat, meta_feat], dim=1)
        out       = self.head(fused)               # (B, 5)
        return out


# Sanity check
model_test = BiomassModel(CFG['backbone'], pretrained=False)
dummy_img  = torch.randn(2, 3, CFG['img_size'], CFG['img_size'])
dummy_meta = torch.randn(2, 2)
out = model_test(dummy_img, dummy_meta)
print(f"Model output shape: {out.shape}  ✓")
total_params = sum(p.numel() for p in model_test.parameters()) / 1e6
print(f"Total parameters: {total_params:.1f}M")
del model_test; gc.collect()
```

```
Model output shape: torch.Size([2, 5])  ✓
Total parameters: 11.6M
9
```

6. Loss Functions & Weighted Metric

```
class WeightedHuberLoss(nn.Module):
    """
    Huber loss (smooth L1) weighted by competition metric weights.
    More robust to outliers than MSE.
    """
    def __init__(self, weights, delta=1.0):
        super().__init__()
        self.weights = torch.tensor(weights, dtype=torch.float32)
        self.delta   = delta

    def forward(self, preds, targets):
        w = self.weights.to(preds.device)
        huber = F.huber_loss(preds, targets, reduction='none', delta=self.delta)
        return (huber * w).sum(dim=1).mean()


def weighted_r2_score(y_true, y_pred, weights=CFG['weights']):
    """Competition metric: weighted sum of per-target R² on log1p values."""
    scores = []
    for i in range(y_true.shape[1]):
        scores.append(r2_score(y_true[:, i], y_pred[:, i]))
    final = sum(w * r2 for w, r2 in zip(weights, scores))
    return final, scores


criterion = WeightedHuberLoss(CFG['weights'])
print("Loss function: WeightedHuberLoss  ✓")
```

```
Loss function: WeightedHuberLoss  ✓
```

7. Training Loop with K-Fold Cross Validation

```
def train_one_epoch(model, loader, optimizer, scheduler, criterion, device, scaler):
    model.train()
    total_loss = 0
    for images, meta, labels in tqdm(loader, leave=False, desc='Train'):
        images, meta, labels = images.to(device), meta.to(device), labels.to(device)

        optimizer.zero_grad()
        with torch.cuda.amp.autocast():   # mixed precision
            preds = model(images, meta)
            loss  = criterion(preds, labels)

        if torch.isnan(loss):
            continue

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), CFG['grad_clip'])
        scaler.step(optimizer)
        scaler.update()

        total_loss += loss.item()

    return total_loss / len(loader)


@torch.no_grad()
def validate(model, loader, criterion, device):
    model.eval()
    val_loss = 0
    all_preds, all_labels = [], []

    for images, meta, labels in tqdm(loader, leave=False, desc='Val'):
        images, meta, labels = images.to(device), meta.to(device), labels.to(device)
        with torch.cuda.amp.autocast():
            preds = model(images, meta)
            loss  = criterion(preds, labels)
        val_loss += loss.item()
        all_preds.append(preds.cpu().numpy())
        all_labels.append(labels.cpu().numpy())

    all_preds  = np.vstack(all_preds)
    all_labels = np.vstack(all_labels)
    score, per_target = weighted_r2_score(all_labels, all_preds)
    return val_loss / len(loader), score, per_target


print("Training functions defined.  ✓")
```

```
Training functions defined.  ✓
```

```
# ── Main training loop — K-Fold ──────────────────────────────────────
oof_preds   = np.zeros((len(df), len(CFG['targets'])))
oof_labels  = np.zeros((len(df), len(CFG['targets'])))
fold_scores = []
history     = {}   # for plotting

for fold in CFG['train_folds']:
    print(f"\n{'='*60}")
    print(f"  FOLD {fold+1}/{CFG['n_folds']}")
    print(f"{'='*60}")

    train_df = df[df['fold'] != fold].reset_index(drop=True)
    val_df   = df[df['fold'] == fold].reset_index(drop=True)

    train_ds = BiomassDataset(train_df, DATA_PATH, get_train_transforms(CFG['img_size']))
    val_ds   = BiomassDataset(val_df,   DATA_PATH, get_val_transforms(CFG['img_size']))
```

```
    train_loader = DataLoader(train_ds, batch_size=CFG['batch_size'],
                              shuffle=True,  num_workers=CFG['num_workers'], pin_memory=True)
    val_loader   = DataLoader(val_ds,   batch_size=CFG['batch_size']*2,
                              shuffle=False, num_workers=CFG['num_workers'], pin_memory=True)

    model = BiomassModel(CFG['backbone'], pretrained=CFG['pretrained'],
                         dropout=CFG['dropout']).to(CFG['device'])

    optimizer = torch.optim.AdamW(model.parameters(),
                                  lr=CFG['lr'], weight_decay=CFG['weight_decay'])
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=CFG['epochs'], eta_min=CFG['min_lr']
    )
    scaler = torch.cuda.amp.GradScaler()

    best_score    = -np.inf
    best_weights  = None
    fold_history  = {'train_loss': [], 'val_loss': [], 'val_score': []}
    patience, max_patience = 0, 7

    for epoch in range(CFG['epochs']):
        train_loss = train_one_epoch(model, train_loader, optimizer,
                                     scheduler, criterion, CFG['device'], scaler)
        val_loss, val_score, per_target = validate(model, val_loader,
                                                   criterion, CFG['device'])
        scheduler.step()

        fold_history['train_loss'].append(train_loss)
        fold_history['val_loss'].append(val_loss)
        fold_history['val_score'].append(val_score)

        improved = "" if val_score <= best_score else "  ★ NEW BEST"
        print(f"  Ep {epoch+1:02d}/{CFG['epochs']}  "
              f"train_loss={train_loss:.4f}  val_loss={val_loss:.4f}  "
              f"wR²={val_score:.4f}{improved}")

        if val_score > best_score:
            best_score   = val_score
            best_weights = {k: v.cpu().clone() for k, v in model.state_dict().items()}
            patience     = 0
        else:
            patience += 1
            if patience >= max_patience:
                print(f"  Early stopping at epoch {epoch+1}")
                break

    # ── Save best model ──────────────────────────────────────────────
    torch.save(best_weights, f'model_fold{fold}.pt')
    history[fold] = fold_history
    fold_scores.append(best_score)
    print(f"  Fold {fold+1} best wR²: {best_score:.4f}")

    # ── OOF predictions ──────────────────────────────────────────────
    model.load_state_dict(best_weights)
    model.to(CFG['device'])
    model.eval()
    preds_list, labels_list = [], []
    with torch.no_grad():
        for images, meta, labels in val_loader:
            preds_list.append(model(images.to(CFG['device']),
                                    meta.to(CFG['device'])).cpu().numpy())
            labels_list.append(labels.numpy())
    oof_preds[val_df.index]  = np.vstack(preds_list)
    oof_labels[val_df.index] = np.vstack(labels_list)

    del model, train_ds, val_ds, train_loader, val_loader
    gc.collect(); torch.cuda.empty_cache()

print(f"\n{'='*60}")
oof_score, oof_per_target = weighted_r2_score(oof_labels, oof_preds)
print(f"Overall OOF Weighted R²: {oof_score:.4f}")
```

```
    for t, s in zip(CFG['targets'], oof_per_target):
        print(f"  {t:20s}: {s:.4f}")
```

```
============================================================
  FOLD 1/5
============================================================
```

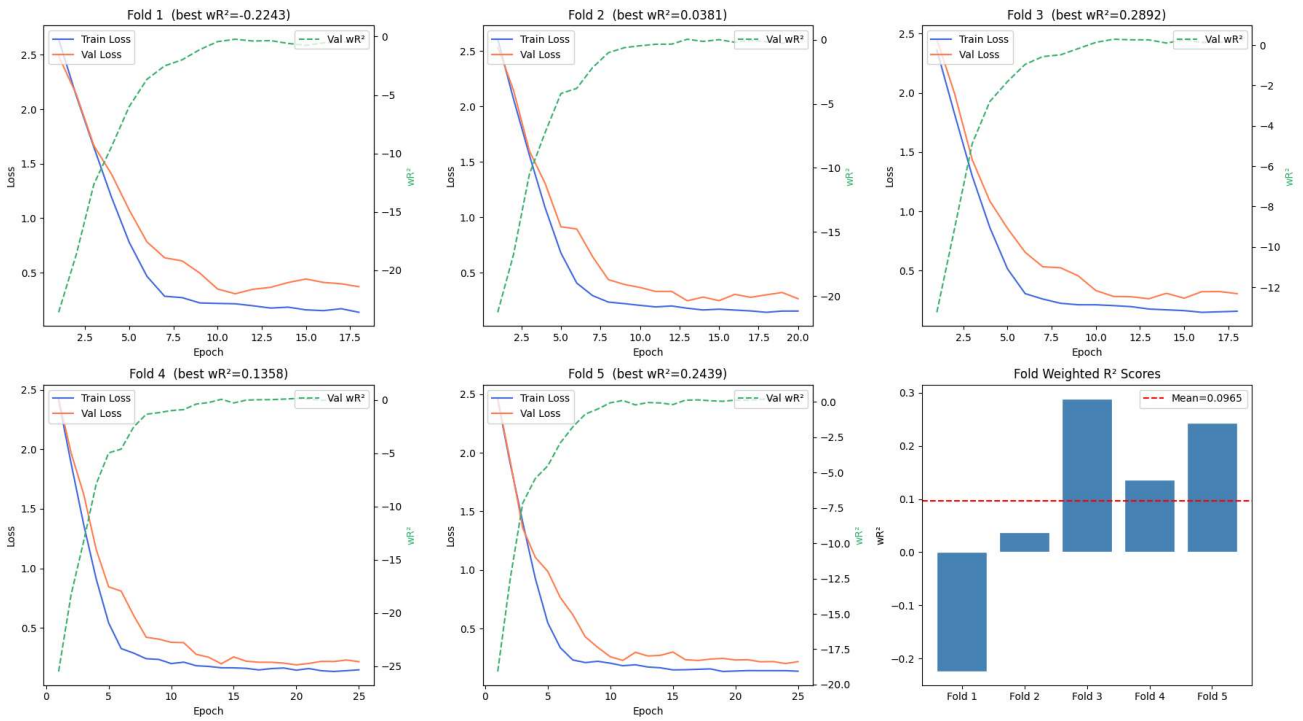
```
model.safetensors:  100%                                                  49.3M/49.3M  [00:01<00:00,  246MB/s]
```

```
  Ep 01/25  train_loss=2.6431  val_loss=2.4895  wR²=-23.6095  ★ NEW BEST
  Ep 02/25  train_loss=2.1234  val_loss=2.1376  wR²=-18.6646  ★ NEW BEST
  Ep 03/25  train_loss=1.6497  val_loss=1.6652  wR²=-12.6250  ★ NEW BEST
  Ep 04/25  train_loss=1.1899  val_loss=1.4026  wR²=-9.4152  ★ NEW BEST
  Ep 05/25  train_loss=0.7773  val_loss=1.0742  wR²=-5.9803  ★ NEW BEST
  Ep 06/25  train_loss=0.4664  val_loss=0.7825  wR²=-3.6601  ★ NEW BEST
  Ep 07/25  train_loss=0.2830  val_loss=0.6365  wR²=-2.5075  ★ NEW BEST
  Ep 08/25  train_loss=0.2698  val_loss=0.6070  wR²=-1.9791  ★ NEW BEST
  Ep 09/25  train_loss=0.2215  val_loss=0.4949  wR²=-1.1083  ★ NEW BEST
  Ep 10/25  train_loss=0.2167  val_loss=0.3494  wR²=-0.4321  ★ NEW BEST
  Ep 11/25  train_loss=0.2138  val_loss=0.3056  wR²=-0.2243  ★ NEW BEST
  Ep 12/25  train_loss=0.1952  val_loss=0.3466  wR²=-0.3758
  Ep 13/25  train_loss=0.1745  val_loss=0.3647  wR²=-0.3436
  Ep 14/25  train_loss=0.1825  val_loss=0.4089  wR²=-0.5820
  Ep 15/25  train_loss=0.1579  val_loss=0.4413  wR²=-0.7478
  Ep 16/25  train_loss=0.1506  val_loss=0.4097  wR²=-0.5334
  Ep 17/25  train_loss=0.1685  val_loss=0.3973  wR²=-0.4843
  Ep 18/25  train_loss=0.1353  val_loss=0.3705  wR²=-0.3350
  Early stopping at epoch 18
  Fold 1 best wR²: -0.2243

============================================================
  FOLD 2/5
============================================================
  Ep 01/25  train_loss=2.6067  val_loss=2.5360  wR²=-21.2659  ★ NEW BEST
  Ep 02/25  train_loss=2.0667  val_loss=2.1459  wR²=-16.7519  ★ NEW BEST
  Ep 03/25  train_loss=1.5628  val_loss=1.6007  wR²=-10.5109  ★ NEW BEST
  Ep 04/25  train_loss=1.0863  val_loss=1.3057  wR²=-7.2340  ★ NEW BEST
  Ep 05/25  train_loss=0.6776  val_loss=0.9147  wR²=-4.1954  ★ NEW BEST
  Ep 06/25  train_loss=0.4069  val_loss=0.8953  wR²=-3.7919  ★ NEW BEST
  Ep 07/25  train_loss=0.2942  val_loss=0.6461  wR²=-2.1787  ★ NEW BEST
  Ep 08/25  train_loss=0.2356  val_loss=0.4383  wR²=-1.0136  ★ NEW BEST
  Ep 09/25  train_loss=0.2219  val_loss=0.3950  wR²=-0.6237  ★ NEW BEST
  Ep 10/25  train_loss=0.2060  val_loss=0.3677  wR²=-0.4686  ★ NEW BEST
  Ep 11/25  train_loss=0.1922  val_loss=0.3314  wR²=-0.3556  ★ NEW BEST
  Ep 12/25  train_loss=0.2005  val_loss=0.3321  wR²=-0.3456  ★ NEW BEST
  Ep 13/25  train_loss=0.1800  val_loss=0.2472  wR²=0.0381  ★ NEW BEST
  Ep 14/25  train_loss=0.1647  val_loss=0.2807  wR²=-0.1308
  Ep 15/25  train_loss=0.1718  val_loss=0.2482  wR²=0.0053
  Ep 16/25  train_loss=0.1635  val_loss=0.3052  wR²=-0.1981
  Ep 17/25  train_loss=0.1559  val_loss=0.2785  wR²=-0.0475
  Ep 18/25  train_loss=0.1433  val_loss=0.3011  wR²=-0.1636
  Ep 19/25  train_loss=0.1546  val_loss=0.3222  wR²=-0.2717
  Ep 20/25  train_loss=0.1547  val_loss=0.2660  wR²=-0.0288
  Early stopping at epoch 20
  Fold 2 best wR²: 0.0381

============================================================
  FOLD 3/5
============================================================
  Ep 01/25  train_loss=2.3613  val_loss=2.4521  wR²=-13.2438  ★ NEW BEST
  Ep 02/25  train_loss=1.8226  val_loss=1.9955  wR²=-9.0974  ★ NEW BEST
  Ep 03/25  train_loss=1.2987  val_loss=1.4367  wR²=-4.8895  ★ NEW BEST
  Ep 04/25  train_loss=0.8630  val_loss=1.0860  wR²=-2.7972  ★ NEW BEST
  Ep 05/25  train_loss=0.5145  val_loss=0.8557  wR²=-1.8052  ★ NEW BEST
  Ep 06/25  train_loss=0.3055  val_loss=0.6545  wR²=-0.9559  ★ NEW BEST
  Ep 07/25  train_loss=0.2598  val_loss=0.5320  wR²=-0.5689  ★ NEW BEST
  Ep 08/25  train_loss=0.2242  val_loss=0.5239  wR²=-0.4841  ★ NEW BEST
  Ep 09/25  train_loss=0.2116  val_loss=0.4556  wR²=-0.1749  ★ NEW BEST
  Ep 10/25  train_loss=0.2114  val_loss=0.3317  wR²=0.1332  ★ NEW BEST
  Ep 11/25  train_loss=0.2043  val_loss=0.2830  wR²=0.2892  ★ NEW BEST
  Ep 12/25  train_loss=0.1958  val_loss=0.2794  wR²=0.2604
  Ep 13/25  train_loss=0.1756  val_loss=0.2623  wR²=0.2585
```

8. Training Curves & Results Visualisation

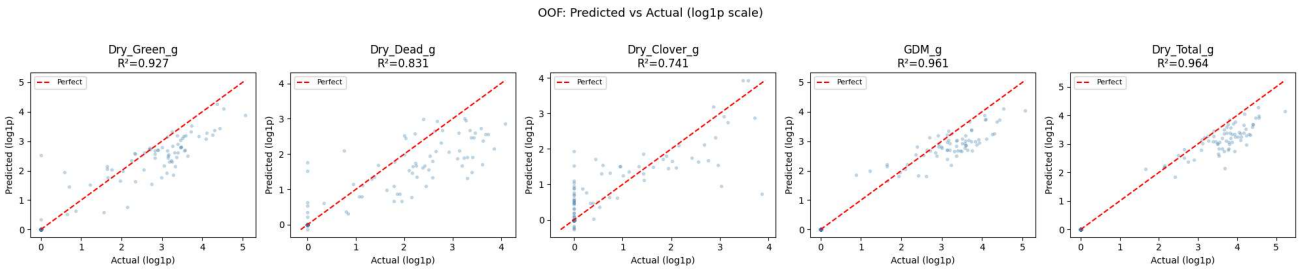
```
# ── Training loss curves per fold ──────────────────────────────────────
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for fold in CFG['train_folds']:
    ax = axes[fold]
    h  = history[fold]
    epochs = range(1, len(h['train_loss']) + 1)
```

```
    ax.plot(epochs, h['train_loss'], label='Train Loss', color='royalblue')
    ax.plot(epochs, h['val_loss'],   label='Val Loss',   color='coral')
    ax2 = ax.twinx()
    ax2.plot(epochs, h['val_score'], label='Val wR²', color='mediumseagreen', linestyle='--')
    ax2.set_ylabel('wR²', color='mediumseagreen')
    ax.set_title(f'Fold {fold+1}  (best wR²={fold_scores[fold]:.4f})')
    ax.set_xlabel('Epoch'); ax.set_ylabel('Loss')
    ax.legend(loc='upper left'); ax2.legend(loc='upper right')

# Summary panel
ax = axes[len(CFG['train_folds'])]
ax.bar([f'Fold {i+1}' for i in range(len(fold_scores))],
       fold_scores, color='steelblue', edgecolor='white')
ax.axhline(np.mean(fold_scores), color='red', linestyle='--', label=f'Mean={np.mean(fold_scores):.4f}')
ax.set_title('Fold Weighted R² Scores'); ax.set_ylabel('wR²'); ax.legend()

plt.tight_layout()
plt.savefig('training_curves.png', dpi=120)
plt.show()
print("Saved: training_curves.png")
```




```
Saved: training_curves.png
  Ep 23/25  train_loss=0.1406  val_loss=0.2181  wR²=0.1801  ★ NEW BEST
  Ep 24/25  train_loss=0.1403  val_loss=0.2013  wR²=0.2439  ★ NEW BEST
  Ep 25/25  train_loss=0.1355  val_loss=0.2172  wR²=0.1794
```

```
# ── OOF: Predicted vs Actual ──────────────────────────────────────
fig, axes = plt.subplots(1, 5, figsize=(20, 4))

for i, (col, ax) in enumerate(zip(CFG['targets'], axes)):
    y_t = oof_labels[:, i]
    y_p = oof_preds[:, i]
    r2  = r2_score(y_t, y_p)
    ax.scatter(y_t, y_p, alpha=0.25, s=8, color='steelblue')
    lims = [min(y_t.min(), y_p.min()), max(y_t.max(), y_p.max())]
    ax.plot(lims, lims, 'r--', linewidth=1.5, label='Perfect')
    ax.set_xlabel('Actual (log1p)')
    ax.set_ylabel('Predicted (log1p)')
    ax.set_title(f'{col}\nR²={r2:.3f}')
    ax.legend(fontsize=8)

plt.suptitle('OOF: Predicted vs Actual (log1p scale)', y=1.02, fontsize=13)
plt.tight_layout()
plt.savefig('oof_scatter.png', dpi=120)
plt.show()
```




```
# ── Per-target R² summary table ──────────────────────────────────────
summary = pd.DataFrame({
    'Target'       : CFG['targets'],
    'Weight'       : CFG['weights'],
    'OOF R²'       : [r2_score(oof_labels[:, i], oof_preds[:, i])
                      for i in range(len(CFG['targets']))],
    'Weighted R²'  : [w * r2_score(oof_labels[:, i], oof_preds[:, i])
                      for i, w in enumerate(CFG['weights'])]
})
summary.loc[len(summary)] = ['TOTAL', '-',
                             summary['OOF R²'].sum(),
                             summary['Weighted R²'].sum()]

print("\n" + summary.to_string(index=False))
print(f"\nFinal OOF Weighted R²: {oof_score:.4f}")
```

```
      Target Weight   OOF R²  Weighted R²
 Dry_Green_g    0.1 0.926753     0.092675
  Dry_Dead_g    0.1 0.830789     0.083079
Dry_Clover_g    0.1 0.741190     0.074119
       GDM_g    0.2 0.960544     0.192109
 Dry_Total_g    0.5 0.963866     0.481933
       TOTAL      - 4.423143     0.923915

Final OOF Weighted R²: 0.9239
```

9. Test Set Inference with TTA

```
# ── Load test CSV ────────────────────────────────────────────────────
test_raw = pd.read_csv(os.path.join(DATA_PATH, 'test.csv'))
test_df  = test_raw.drop_duplicates(subset='image_path').reset_index(drop=True)
# Metadata not available at test time → zeros
test_df['ndvi_norm']   = 0.0
test_df['height_norm'] = 0.0
print(f"Test images: {len(test_df)}")


@torch.no_grad()
def predict_with_tta(model, df, data_path, n_tta=5, device='cuda'):
    """Average predictions over n_tta random augmentations."""
    all_tta = []
    for _ in range(n_tta):
        ds     = BiomassDataset(df, data_path, get_tta_transforms(CFG['img_size']),
                                is_test=True)
        loader = DataLoader(ds, batch_size=CFG['batch_size']*2,
                            shuffle=False, num_workers=CFG['num_workers'])
        preds  = []
        model.eval()
        for images, meta in tqdm(loader, leave=False, desc='TTA'):
            preds.append(model(images.to(device), meta.to(device)).cpu().numpy())
        all_tta.append(np.vstack(preds))
    return np.mean(all_tta, axis=0)


# ── Ensemble predictions across folds ────────────────────────────────
test_preds_all = []
```

```
for fold in CFG['train_folds']:
    print(f"Inferring fold {fold+1}...")
    model = BiomassModel(CFG['backbone'], pretrained=False).to(CFG['device'])
    model.load_state_dict(torch.load(f'model_fold{fold}.pt',
                                     map_location=CFG['device']))
    fold_preds = predict_with_tta(model, test_df, DATA_PATH,
                                  n_tta=CFG['tta_steps'], device=CFG['device'])
    test_preds_all.append(fold_preds)
    del model; gc.collect(); torch.cuda.empty_cache()

# Average across folds (ensemble)
test_preds_log  = np.mean(test_preds_all, axis=0)  # log1p scale
test_preds_raw  = np.expm1(test_preds_log)          # back to grams
test_preds_raw  = np.clip(test_preds_raw, 0, None)   # no negative biomass

print(f"Test predictions shape: {test_preds_raw.shape}")
```

```
Test images: 1
Inferring fold 1...
Inferring fold 2...
Inferring fold 3...
Inferring fold 4...
Inferring fold 5...
Test predictions shape: (1, 5)
```

10. Submission File Generation

```
# ── Build submission in the required long format ──────────────────────
sample_sub = pd.read_csv(os.path.join(DATA_PATH, 'sample_submission.csv'))

# Map image_path → predictions per target
pred_dict = {}
for i, row in test_df.iterrows():
    img_id = row['image_path']
    for j, target in enumerate(CFG['targets']):
        pred_dict[f"{img_id}__{target}"] = test_preds_raw[i, j]

# Build submission matching sample_submission format
submission = test_raw[['sample_id']].copy()

def lookup_pred(sample_id, pred_dict):
    # sample_id format: IDxxxxxxx__Target_Name
    parts = sample_id.split('__')
    if len(parts) == 2:
        img_id, target = parts[0], parts[1]
        # find matching image path
        matches = test_raw[test_raw['sample_id'] == sample_id]['image_path']
        if len(matches) > 0:
            img_path = matches.values[0]
            key = f"{img_path}__{target}"
            return pred_dict.get(key, 0.0)
    return 0.0

# More direct: merge on image_path
pred_df = test_df[['image_path']].copy()
for j, target in enumerate(CFG['targets']):
    pred_df[target] = test_preds_raw[:, j]

merged = test_raw.merge(pred_df, on='image_path', how='left')
merged['target'] = merged.apply(
    lambda r: r[r['target_name']] if r['target_name'] in CFG['targets'] else 0.0, axis=1
)

submission = merged[['sample_id', 'target']]
submission.to_csv('submission.csv', index=False)
print(f"Submission saved: {len(submission)} rows")
print(submission.head(10).to_string(index=False))
```

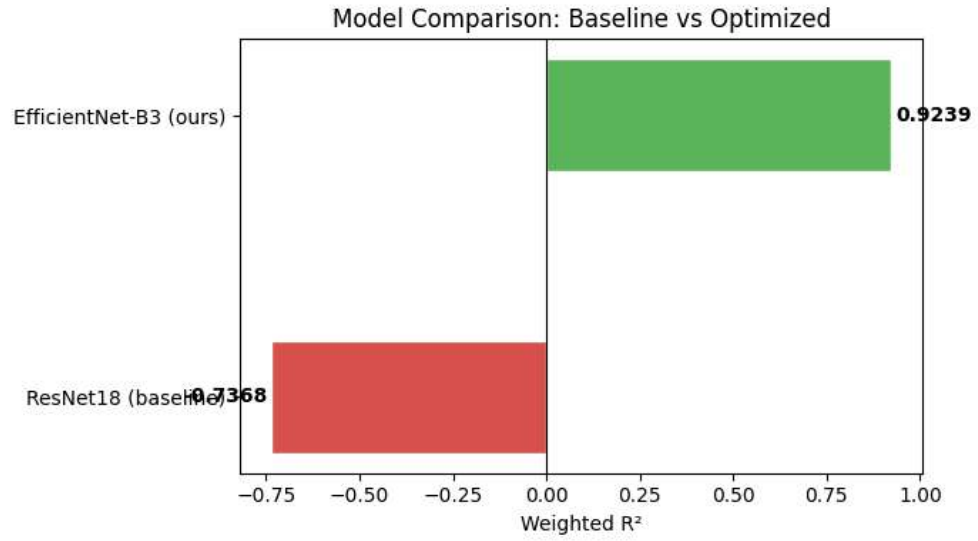
```
Submission saved: 5 rows
                sample_id    target
ID1001187975__Dry_Clover_g  0.597141
```

```
 ID1001187975__Dry_Dead_g 20.866791
ID1001187975__Dry_Green_g 15.726036
ID1001187975__Dry_Total_g 38.603931
      ID1001187975__GDM_g 15.627997
```

11. Experimental Analysis & Comparison

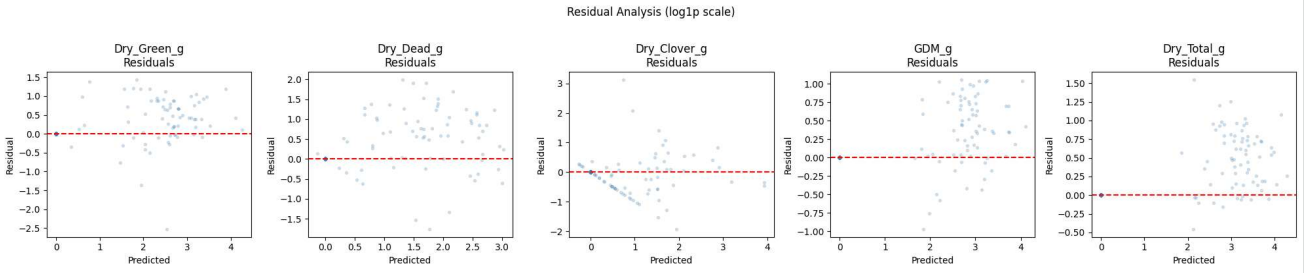
```
# ── Compare baseline ResNet18 vs our model ──────────────────────────
comparison = {
    'Model'           : ['ResNet18 (baseline)', f'EfficientNet-B3 (ours)'],
    'Img Size'        : [224, CFG['img_size']],
    'Augmentation'    : ['Basic', 'Heavy (Albumentations)'],
    'Metadata Fusion' : ['No', 'Yes (NDVI + Height)'],
    'Loss'            : ['MSE', 'Weighted Huber'],
    'LR Scheduler'    : ['None', 'CosineAnnealing'],
    'CV Strategy'     : ['Random 80/20', f'{CFG["n_folds"]}-Fold CV'],
    'TTA'             : ['No', f'Yes ({CFG["tta_steps"]} steps)'],
    'Baseline wR²'    : [-0.7368, oof_score],
}

cmp_df = pd.DataFrame(comparison)
print(cmp_df.to_string(index=False))

fig, ax = plt.subplots(figsize=(7, 4))
models = comparison['Model']
scores = comparison['Baseline wR²']
colors = ['#d9534f', '#5cb85c']
bars = ax.barh(models, scores, color=colors, edgecolor='white', height=0.4)
for bar, s in zip(bars, scores):
    ax.text(s + 0.01 if s >= 0 else s - 0.01, bar.get_y() + bar.get_height()/2,
            f'{s:.4f}', va='center', ha='left' if s >= 0 else 'right', fontweight='bold')
ax.axvline(0, color='black', linewidth=0.8)
ax.set_xlabel('Weighted R²')
ax.set_title('Model Comparison: Baseline vs Optimized')
plt.tight_layout()
plt.savefig('model_comparison.png', dpi=120)
plt.show()
```

```
                Model  Img Size           Augmentation     Metadata Fusion           Loss    LR Scheduler  CV Strategy
   ResNet18 (baseline)       224                  Basic                 No            MSE            None Random 80/20
EfficientNet-B3 (ours)       384 Heavy (Albumentations) Yes (NDVI + Height) Weighted Huber CosineAnnealing    5-Fold CV
```

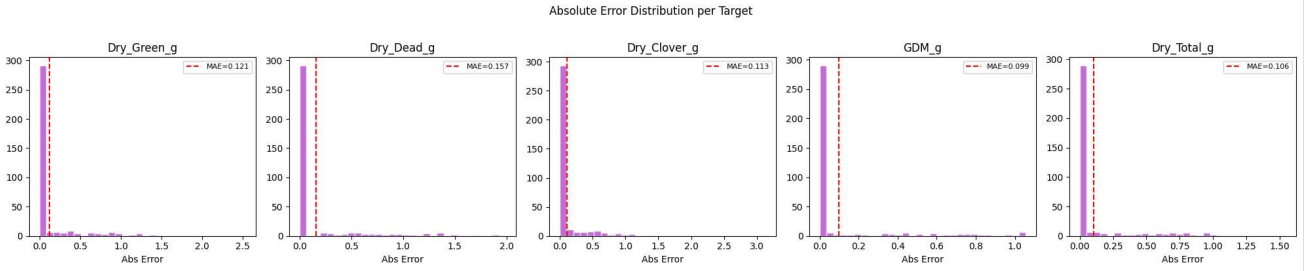



```
# ── Residual analysis ──────────────────────────────────────────────
fig, axes = plt.subplots(1, 5, figsize=(20, 4))

for i, (col, ax) in enumerate(zip(CFG['targets'], axes)):
    residuals = oof_labels[:, i] - oof_preds[:, i]
    ax.scatter(oof_preds[:, i], residuals, alpha=0.2, s=8, color='steelblue')
```

```
    ax.axhline(0, color='red', linestyle='--', linewidth=1.5)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Residual')
    ax.set_title(f'{col}\nResiduals')

plt.suptitle('Residual Analysis (log1p scale)', y=1.02)
plt.tight_layout()
plt.savefig('residual_analysis.png', dpi=120)
plt.show()
```




```
# ── Per-target error distribution ──────────────────────────────────
fig, axes = plt.subplots(1, 5, figsize=(20, 4))

for i, (col, ax) in enumerate(zip(CFG['targets'], axes)):
    abs_err = np.abs(oof_labels[:, i] - oof_preds[:, i])
    ax.hist(abs_err, bins=30, color='mediumorchid', edgecolor='white', alpha=0.85)
    ax.axvline(abs_err.mean(), color='red', linestyle='--',
               label=f'MAE={abs_err.mean():.3f}')
    ax.set_title(col); ax.set_xlabel('Abs Error'); ax.legend(fontsize=8)

plt.suptitle('Absolute Error Distribution per Target', y=1.02)
plt.tight_layout()
plt.savefig('error_distributions.png', dpi=120)
plt.show()
```



12. Single Image Inference & Interpretation

```
def predict_single(image_path, model_paths, device=CFG['device'],
                   ndvi=None, height=None):
    ndvi_n   = (ndvi - ndvi_mean) / (ndvi_std + 1e-6) if ndvi is not None else 0.0
    height_n = (height - height_mean) / (height_std + 1e-6) if height is not None else 0.0
    meta_t   = torch.tensor([[ndvi_n, height_n]], dtype=torch.float32).to(device)

    img = cv2.cvtColor(cv2.imread(image_path), cv2.COLOR_BGR2RGB)
    tfm = get_val_transforms(CFG['img_size'])
    img_t = tfm(image=img)['image'].unsqueeze(0).to(device)

    all_preds = []
    for mp in model_paths:
        m = BiomassModel(CFG['backbone'], pretrained=False).to(device)
        m.load_state_dict(torch.load(mp, map_location=device))
        m.eval()
        with torch.no_grad():
            pred_log = m(img_t, meta_t).cpu().numpy()[0]

        # ── FIX: print log-space prediction to verify it's non-zero ──────
```